sage labs

Sage Mobility v2 Platform
Application Architecture & Best Practices

Introduction

This document outlines the platform architecture and best practices for developing with the Sage Mobility v2 Platform. It is assumed that developers have intermediate skills in JavaScript and familiarity with Titanium, Alloy, and mobile development.

Architecture

The mobility platform functions as a single-window application using Titanium with the Alloy MVC framework. A single-window application uses a single JavaScript context simplifying data access and sharing. The platform architecture is composed of several components including Controllers, Views, Models, and Interactivity.

Architecture: Controllers

All applications begin via an initial *"index"* view controller *(controllers/index.js)*. This controller can handle login or other initialization required prior to using the application. This controller also initializes the core application *(controllers/core/application.js).*

Application Initialization Steps:

1. Create single window.
2. Create and initialize menu *(controllers/core/menu.js).*
    a. Menu creates several event listeners for menu interactivity.
    b. Menu adds itself to the application window.
3. Creates and initializes initial view controller.
4. Opens initial view.

All controllers inherit from the base controller *(controllers/core/base.js)*. The base controller provides layout and navigation. The base controller's "*init"* method should not be overridden as it handles required setup for any controller.

Upon creation, the controllers setup the navigation options including setting the title and visible buttons. After setting the navigation options, the controller should add itself to the *"content"* view. Finally, the base controller's "*init"* method is executed.

After a controller is created and initialized, the current view is set to the new view controller while the previous view controller is pushed onto the previous stack of views.

Architecture: Views

Views are handled via Alloy's XML markup with Titanium style sheets. Every view is associated with a controller. Views have one of several possible view modes. This can be set when the application opens the view as well as within the controller. These modes determine how to display and what to include in the navigation bar.

View Modes	Display Navigation Bar	Display Standard Buttons
nav	Yes	Yes
simple	Yes	No
blank	No	No

All controllers inherit a set of default view options. The controller can override these options as well as include additional options. The default options include the following.

View Options	Default Value	View Impact
topLevel	false	Display back button
viewMode	nav	Display the navigation bar
transition	slideInFromRight	Slide in from right
duration	150	Transition will be 150ms

The three transitions currently available are slideInFromRight, slightInFromLeft, and none. The *"slideInFromRight"* and *"slightInFromLeft"* transitions animate the new view sliding in from the specified direction with the previous view sliding out in the opposite direction. The *"none"* transition replaces views without any animation.

Architecture: Models

The data models are defined using the Alloy definition structure including a SQL adapter for local storage. In order to support Backbone v1.1.2, all data models must extend the Collection prototype and override the fetch method for Collections.

```
fetch: function(options) {
      options = options ? _.clone(options) : {};
      options.reset = true;
      return Backbone.Collection.prototype.fetch.call(this, options);
},
```

One can also extend the Model prototype including a transform function to setup properties for display. For example, any property that represents currency as a "real" data type can be formatted as currency and returned as a "String" for display purposes.

Architecture: Interactivity

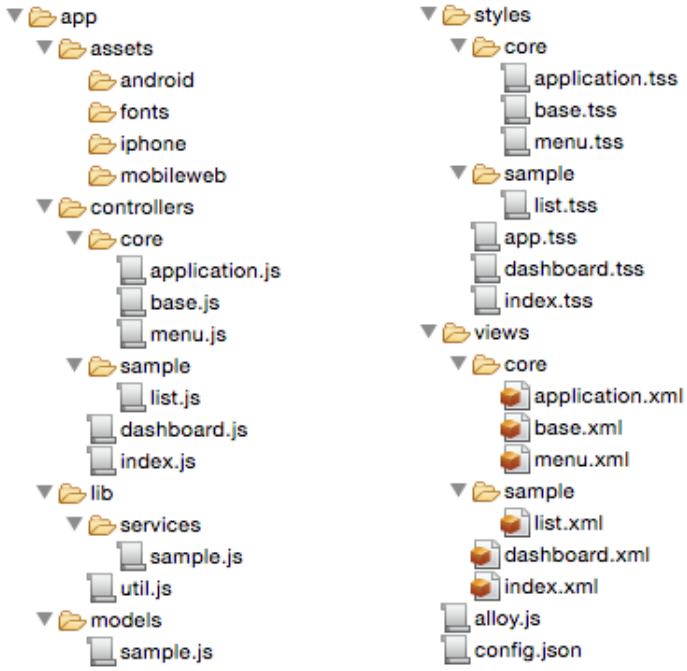
The main menu is implemented as a drawer that is accessible via the menu button or by swiping on a mobile device *(controllers/core/menu.js)*.

The menu is contained in a special table view. Each row in the view has several properties that indicate the functionality of that menu item. For example, if the *"id"* property is "*inactive*" then that menu item is not active and has no action.

Upon selecting a menu item, an event handler will open the new controller with the name of controller specified in the "*controller"* custom property. The *"topLevel"* property is set to true. Finally, the *"id"* property of the row is used as a unique identifier in the previous view controller stack.

Architecture: Application Organization

An application built using the platform should follow a standard Titanium Alloy application structure.



Best Practices

Application development should adhere to the following best practices and recommendations. These guidelines are mandatory for developers to follow unless otherwise noted. The guidelines categories include JavaScript, Titanium, Cross-Platform Mobile Development, Data Access, and User Interface.

Best Practices: JavaScript

With regards to JavaScript, developers should follow the Google JavaScript style guide.

http://google.github.io/styleguide/javascriptguide.xml

In addition, these best practices should be followed.

- Keep use of global variables to a minimum using one of these methods.
    - First, use a CommonJS module to store and share global variables.
    - Secondly, use no more than 2 to 3 global variables accessed via *"Alloy.Globals"*.
    - Global variables in this case do not include configuration parameters.
- Follow the JavaScript naming conventions as given in the above style guide.
    - Do not use reserved words as identifiers using the current list of reserved words provided by Appcelerator.

      http://docs.appcelerator.com/platform/latest/#!/guide/Reserved_Words.
    - Do not use double underscore prefixes on variable or function names as this can conflict with Alloy.
- Use CommonJS modules to build reusable libraries.
    - For libraries, use *"exports."* notation to declare public methods.
    - For classes, use *"module.exports"* notation to declare the class as public.
    - CommonJS modules do not have access to any variables in the global namespace on all devices.
        - All variables required by a module should be defined within the module or as a parameter to a method.
- Modules are only loaded once and cached. Only one instance exists per context.
    - To include modules, use the following code.

      ```
      if (typeof m === 'undefined') { var m = require('m'); }
      ```
    - Avoid loading scripts until they are absolutely needed.
- Use event handlers and callbacks for communication between components.
    - Use callbacks over event handlers where possible because callbacks perform better than event handlers in most cases.
    - Avoid global event handlers except for when handling global objects.
    - Avoid using local objects in global event listeners.
    - Do not name custom events with spaces.

Best Practices: Titanium

When developing with Titanium and Alloy, developers should follow the guidelines given by Appcelerator.

http://docs.appcelerator.com/platform/latest/#!/guide/Best_Practices_and_Recommendations

http://docs.appcelerator.com/platform/latest/#!/guide/Alloy_Best_Practices_and_Recommendations

In addition, the following best practices should be implemented.

- Group related controllers/views/styles into separate sub-directories.
- Set local variables to avoid calling native methods each time you request the value of a device-related property via "*Titanium.Platform"* namespace methods.
- Destroy all windows and nullify all objects when done.
- Do not create windows using "*url"* property.
- Use "*ListViews"* over "*TableViews"* for lists larger than 20 items because "*ListView"* performs better and does not requiring loading and rendering the entire data set at once.
- Do not add or extend any object in Titanium namespace. Create wrappers around proxy objects in order to extend objects and add data.
- Configuration should be centralized via the following methods.
  - Use "*config.json"* to handle properties that do not change at run-time but vary with different environments.
  - Use "*tiapp.xml"* to handle properties that do not change at run-time and do not vary by platform or environment.
  - Use "*alloy.js"* to handle properties that are global variables but may be changed at run-time.

Best Practices: Cross-Platform Mobile Development

To make cross-platform mobile development easier, use these recommendations.

- Focus on Android first as it has more quirks than iOS.
- Keep in mind that some UI components and properties are platform specific. The Appcelerator API documentation identifies supported platforms for each class, property, method, and event.
- Use compiler directives in Alloy, such as "*OS_IOS"* and "*OS_ANDROID"*, for switching platform code.
- Use platform specific files when code varies greatly between platforms.
- Don't store sensitive data in non-JavaScript files because these files are stored as plain text files on the device.

- Test applications on both iOS (v8.x) and Android (v4.1) via emulators. Test applications on physical devices when available. Testing different form factors, such as handheld or tablet, is specific to the application and project.

Best Practices: Data Access

When exchanging data with back-end systems, follow these guidelines.

- Keep data models as flat as possible. Data duplication between different local data models is acceptable but may have synchronization issues.
- Store data locally and present local data upon rendering a view. As view is rendering, fetch updated data from back-end systems.
- All data required to render a single view should be available via 1 call to the back-end system. Storing data in the back-end system should be accomplished in 1 - 2 calls to the back-end system.

Best Practices: User Interface

When developing the user interface, developers should follow these guidelines.

- Use density pixels *("dp")* for layout and scaled pixels *("sp")* for font sizes.
- Use Alloy XML Views to provide the layout structure of components. Use Titanium Style Sheets to specify layout locations and style of components.
- Styles are defined at several levels: markup element (*"Label"*), class attribute (*".labels"*), and id attribute (*"#mylabel"*). The markup elements are overridden by both class and id attributes. Class attributes are overridden by id attributes only.
- Keep class-based styles small for reuse and include multiple styles in the markup class property separated by spaces. These styles are applied from left to right.
- Use global styles in the *"app.tss"* file minimally. If the number of global styles required continues to increase, consider using a global theme instead.
- Styles are applied starting with global styles/themes that are overridden by view specific styles/themes. Finally, any style attribute set within the markup will override all other values set via global or view styles.
- Colors should be defined in global variable constants that are used in all code and style sheets.
- Fonts, including family, size, and weight should be defined as separate global class-based styles reused by all views.
- Icons should be handled via icon fonts and should be defined as global class-based styles. These styles should define both the "text" and "title" properties for use by Labels and Buttons respectively.